《算法分析与设计》
课后作业

作业编号 作业 5

学　　号 _____

姓　　名 _____

专　　业 _____

学　　院 _____

二 0 二四年五月

第 5 章 动态规划算法

题目 1

某公司进行组合投资，把钱同时投入到多个项目，并希望获取最大的收益。在已知该公司希望投资的最大金额数和投资的项目数的情况下，请编写程序计算其所能获得的最大收益。

输入：

输入的第一行包含两个整数 n 和 m，分别表示该公司的投资资金数和希望投资的项目数。其后的 n 行，每行有 m 个整数，分别表示将金额 1、2、…、n 单独投资到 m 个项目时的收益（投资为 0 时 m 个项目的收益均为 0）。每次组合投资必须将 n 用完，组合投资中某个项目的投资额可以为 0。

输出：

输出一行，表示该公司进行组合投资所能获得的最大收益。

样例输入：

3 2

3 2

3 5

4 7

样例输出：

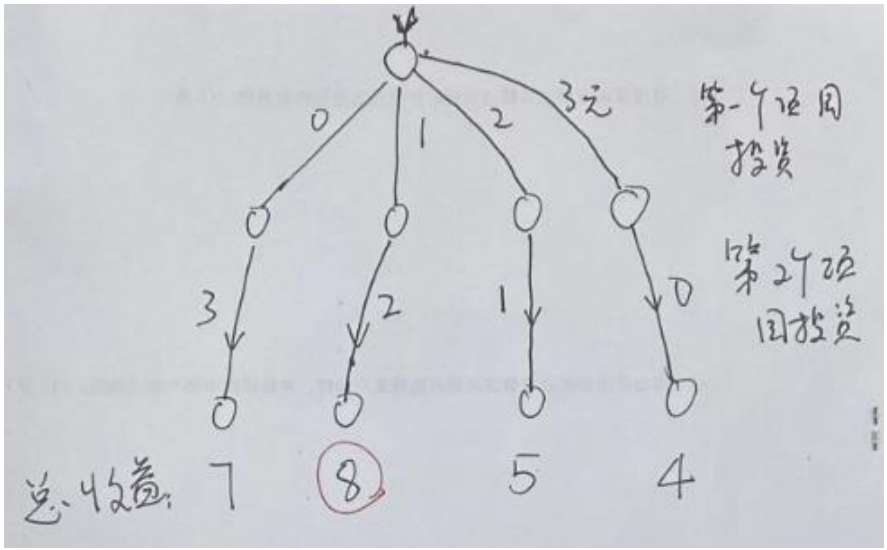
8

要求：

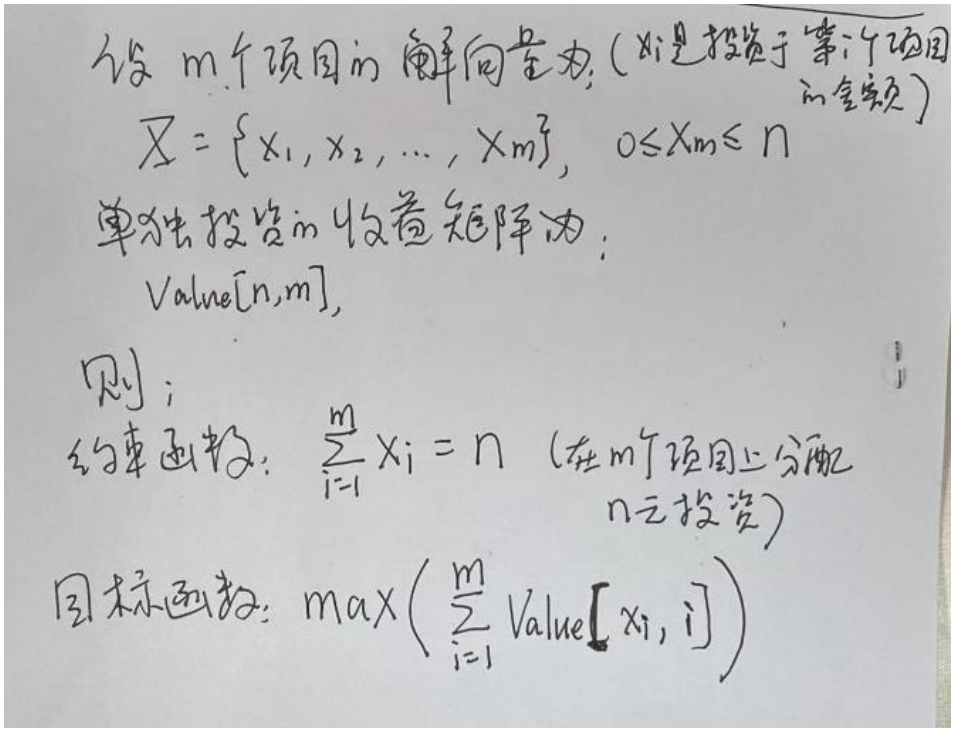
（1）针对该测试用例，画出采用回溯法求解问题的搜索空间树，并写出目标函数和约束条件。

参考答案：

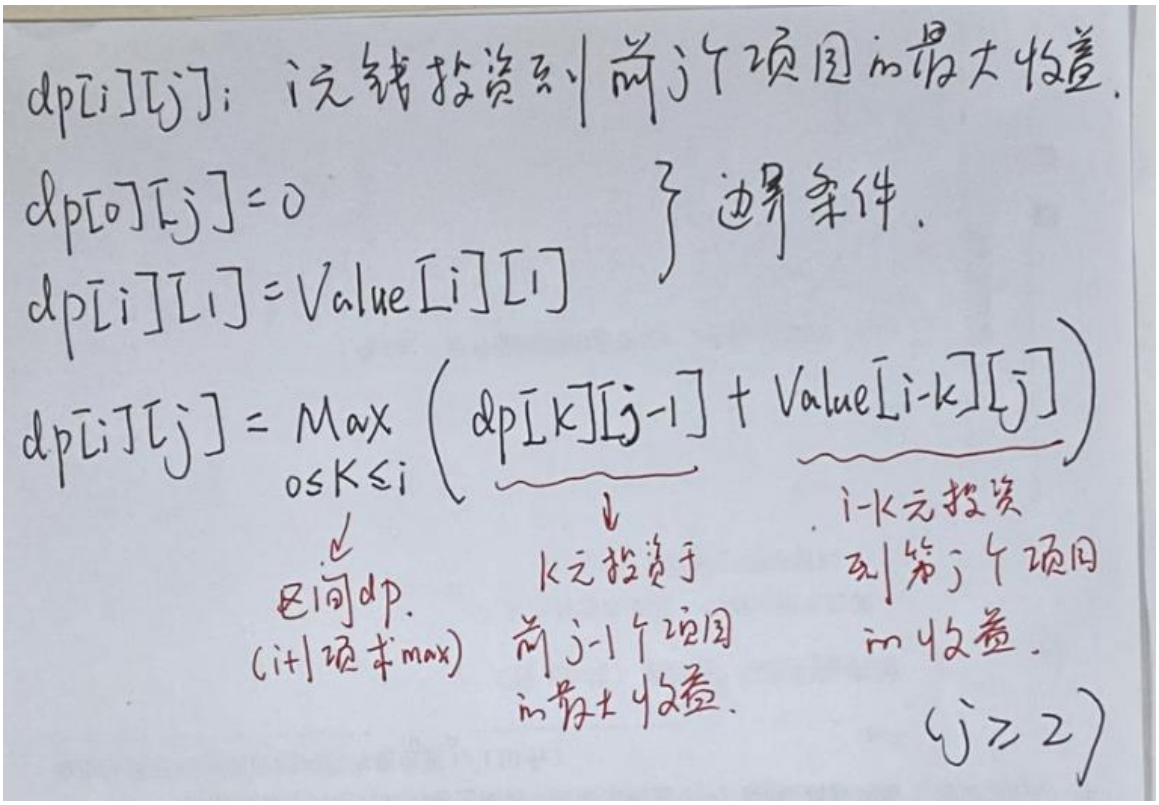
解空间树



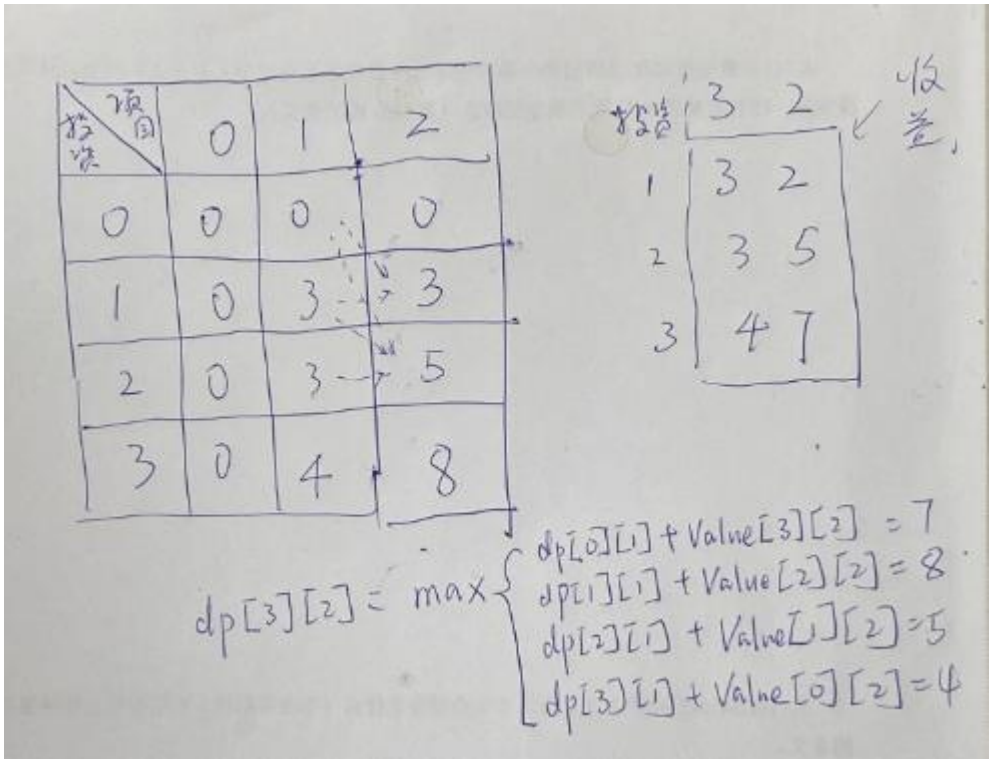
约束条件和目标函数：



（2）写出采用动态规划法求解该问题的递推方程和边界条件。



（3）用动态规划法分析样例输入的求解过程，手工计算 dp 数组的值。



（4）采用动态规划法编写程序实现上述题目要求。

略

题目 2

有一天美羊羊正在草地上玩耍，突然天上开始落金币，这些金币掉落的范围在一个固定的水平区域内，但这些金币一旦掉落到地上就消失了，因此美羊羊只有不断地移动并从空中接住这些金币才能得到它们。假设金币掉落的位置为 0 开始到 10 这 11 个位置，美羊羊开始时站在第 5 个位置，它可以以每秒 1 个位置的速度左右移动到相邻的位置，并接住掉落的金币。请问美羊羊最多能接住多少个金币？假设它一旦接住这些金币就不会掉落到地上。



输入：

输入数据有多组。每组数据的第一行为一个正整数 n（0<n<100000），表示有 n 个金币掉落在这个区域。在接下来的 n 行中，每行有两个整数 x，t（0<t<100000），表示在第 t 秒有一个金币掉落在 x 的位置上。同一秒钟在同一位置上可能掉下多个金币，n=0 时输入结束。

输出：

每一组输入数据对应一行输出。输出一个整数 m，表示美羊羊最多可能接到 m 个金币。

样例输入：

6

5 1

4 1

6 1

7 2

7 2

8 3

0

样例输出：

4

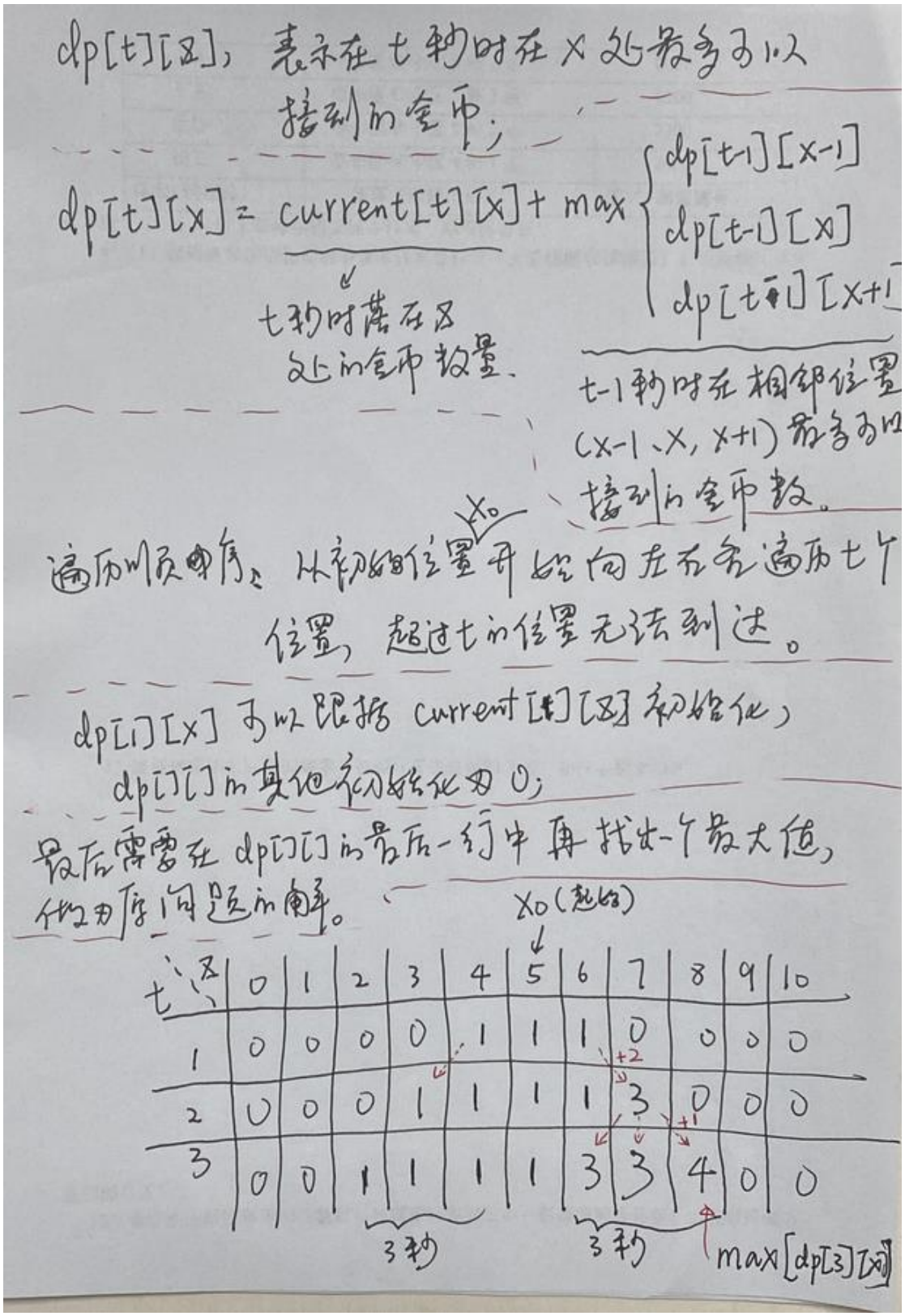
要求：

（1）写出采用动态规划法求解该问题的递推方程和边界条件。

（2）用动态规划法分析样例输入的求解过程，手工计算 dp 数组的值。

参考答案：

如下所示

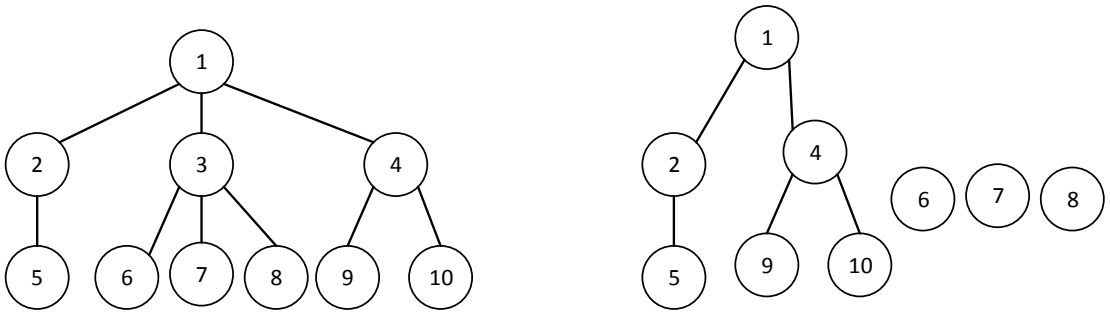


（3）采用动态规划法编写程序实现上述题目要求。

略

题目 3

定义树中某结点的平衡为删除该结点后所形成的森林中各树结点数的最大值。如下图左边的树，如果删除其中的一个结点 3，则该树变成右边的森林，也就是多棵树，这些树的结点数分别为 6，1，1，1，其最大值为 6，因此结点 3 的平衡为 6。



在给定一棵树的情况下，求该树中所有结点的最小平衡数。

输入：

输入的第一行是一个整数 n（(1 <= n<= 20)，表示测试数据的组数。每一组测试数据的第一行是一个整数 m (1 <= m <= 20,000)，表示树中边的数量，其后的 m 行，每一行包含两个整数，表示树中相连的两个结点。树中结点的编号从 1 开始编号。

输出：

对于每一组测试数据输出一行，表示该树中所有结点的平衡数的最小值。

输入样例：

1

9

1 2

1 3

1 4

2 5

3 6

3 7

3 8

4 9

4 10

输出样例：

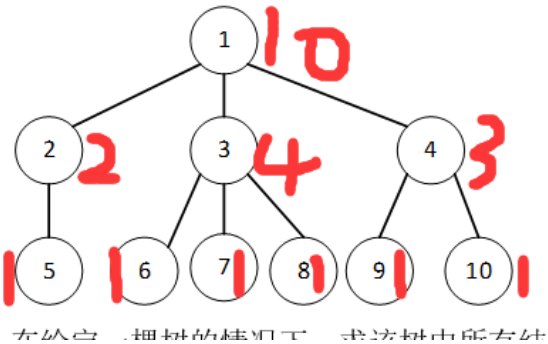
4

要求：

（1）分析样例输入的求解过程，手工得到计算结果。

（2）编写程序实现上述题目要求。

参考答案



1、采用 dfs 计算出每个节点的子树中的总结点数（含当前节点，如上图），保存在一个数组（如 nodes）中。

2、求树中每个结点 i 的平衡值，公式如下：

dp[i] = max(nodes[a], nodes[b],…, nodes[root] – nodes[i])；a、b……..等表示当前结点 i 的后代结点，root 表示树根。

注意：当删除一个结点后，该结点的每个下级分支会变成一个子树；此外原有的树去除该结点，会变成另外一个子树（如果删除的是根结点，则没有该子树）。例如要求上述 3 号结点的平衡值，把它删除后，会产生 3 个下级子树（结点数均为 1），以及一个结点数为 6（10-4）的子树，因此 3 号结点的平衡为 6。同理可以求出所有结点的平衡，保存在另一个数组中。

3、计算所有结点的平衡数的最小值。

（样例输入中结点 1 的平衡数为 4，这是所有结点的平衡的最小值）。

参考代码：

```
#include <iostream>
#include <cstring>
#include <algorithm>
using namespace std;
const int maxm = 2e4 + 5;
int head[maxm], vex[maxm], nxt[maxm], idx;
int nodes[maxm], dp[maxm];
bool root[maxm];
int ans = 0x3f3f3f3f;
inline void addedge(int x, int y){ //这里采用“链式前向量”保存图，是空间效率最高的方式
    idx++;
    vex[idx] = y;
    nxt[idx] = head[x];
    head[x] = idx;
}
```

```
int dfs1(int curnode){
    int sum = 1;                                    // curnode 本身就是一个结点
    for(int i = head[curnode]; i; i = nxt[i]){    //求 curnode 的子树的总结点数
        int adjv = vex[i];
        sum += dfs1(adjv);
    }
    nodes[curnode] = sum;
    return sum;
}

void dfs2(int curnode, int& treeroot){
    int max_nodes = 0;
    for(int i = head[curnode]; i; i = nxt[i]){    //求 curnode 的最大子树的节点数
        int adjv = vex[i];
        max_nodes = max(max_nodes,    nodes[adjv]);
    }
    //求整棵树去掉 curnode 子树后剩余的结点数
    max_nodes = max(max_nodes,    nodes[treeroot] - nodes[curnode]);
    dp[curnode] = max_nodes;
    for(int i = head[curnode]; i; i = nxt[i]){
        int adjv = vex[i];
        dfs2(adjv,    treeroot);    //递归 curnode 的子树
    }
}

int run_case(){
    memset(head, 0, sizeof(head));
    memset(root, 1, sizeof(root));
    memset(dp, 0x3f, sizeof(dp));
    memset(nodes, 0, sizeof(nodes));
    int n, m;
    cin >> n >> m;
    for(int i = 0; i < m; i++){
        int x, y;
        cin >> x >> y;
        root[y] = false;          //x->y,把 y 标记成不是树根
        addedge(x, y);
    }
    int treeroot;
    for(int i = 1; i <= n; i++){//找到树根编号
        if(root[i]){
            treeroot = i;
            break;
        }
```

```
    }
    dfs1(treeroot);                 //求每棵子树的总结点树,从树根开始搜
    dfs2(treeroot, treeroot);

    cout << *min_element(&dp[1], &dp[n]) << "\n";

    return 0;
}
```

课堂练习 1：最大上升子序列长度

- 给定一个整数序列（$a_1, a_2, \ldots, a_n$），要求选出尽量多的元素，使这些元素按其相对位置单调递增。
  - 输入：一个整数序列
  - 输出：最大单调递增子序列的长度
  - 样例输入：5 8 9 2 3 1 7 4 6
  - 样例输出：4（即：2 3 4 6）
- （1）写出采用动态规划法求解该问题的 dp 数组的含义和递推方程。
- （2）用动态规划法分析样例输入的求解过程，手工计算 dp 数组的值。

答案

定义 $dp[i]$ 为考虑前 i 个元素，以第 i 个数字结尾的最长上升子序列的长度，**注意 $nums[i]$ 必须被选取。**

我们从小到大计算 dp 数组的值，在计算 $dp[i]$ 之前，我们已经计算出 $dp[0 \ldots i-1]$ 的值，则状态转移方程为：

$$dp[i] = \max(dp[j]) + 1, \text{其中 } 0 \le j < i \text{ 且 } num[j] < num[i]$$

最终答案为：$\mathrm{LIS}_{length} = \max(dp[i]), \text{其中 } 0 \le i < n$

样例输入：5 8 9 2 3 1 7 4 6（长度为 9）

样例输入对应的dp数组：

下标 i	1	2	3	4	5	6	7	8	9
Dp[i]	1	2	3	1	2	1	3	3	4

在dp数组中再求一个最大值，结果为 4（原问题的解）

课堂练习 2：删除字符的最少次数

给定两个字符串 s1 和 s2，现在希望仅仅通过删除字符的操作，使两个字符串变成相同的字符串。每次可以删除任意一个字符串中的一个字符。用动态规划算法计算所需删除操作的最少次数。

输入描述：

输入 2 行，第一行是第 1 个字符串 s1，第二行是第 2 个字符串 s2。

输出描述：

输出 1 行，表示所需删除操作的最少次数。

样例输入：

seal

eat

样例输出：

3

答案

1. 确定 dp 数组（dp table）以及下标的含义：

dp[i][j]：以 i-1 为结尾的字符串（word1）和以 j-1 为结尾的字符串（word2）要达到字符串相等，所需要删除元素的最少次数。

2. 确定递推公式：

当 word1[i - 1]与 word2[j - 1]相同的时候，dp[i][j] = dp[i - 1][j - 1]。

当 word1[i - 1] 与 word2[j - 1]不相同的时候，有 3 种情况：

情况 1：删 word1[i - 1]，最少操作次数为 dp[i - 1][j] + 1

情况 2：删 word2[j - 1]，最少操作次数为 dp[i][j - 1] + 1

情况 3：同时删 word1[i - 1]和 word2[j - 1]，操作的最少次数为 dp[i - 1][j - 1] + 2

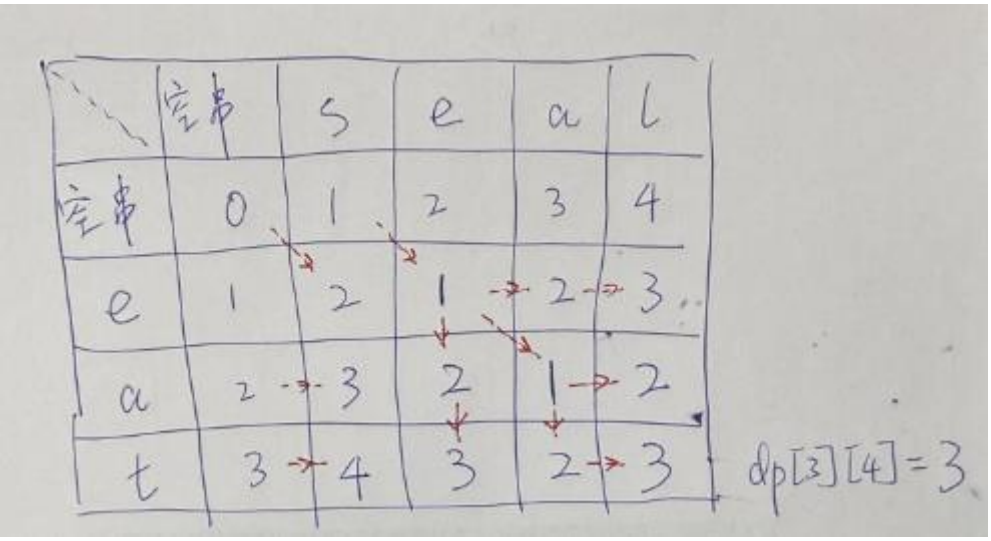
所以当 word1[i - 1] 与 word2[j - 1]不相同的时候，递推公式：

dp[i][j] = min({dp[i - 1][j - 1]+2, dp[i-1][j] + 1, dp[i][j - 1] + 1})。

3. dp 数组初始化：

dp[i][0]：word2 为空字符串，以 i-1 为结尾的字符串 word2 要删除多少个元素，才能和 word1 相同呢，很明显 dp[i][0] = i。同理 dp[0][j]=j。

样例输入时 dp 数组的求解过程：



答案 2：

本题也可以先求出两个字符串的最长公共子序列（LCS）长度，然后用两个字符串的长度之和减去最长公共子序列长度的两倍，即为所需的最少删除次数。

	空串	**S**	**E**	**A**	**L**
空串	0	0	0	0	0
E	0	0	1	1	1
A	0	0	1	2	2
T	0	0	1	2	2

即 LCS Length=2，所以原问题解=（4-2）+（3-2）=3

实验题目：字符串相似度

参考答案：

设 f[i][j]表示序列 1 中前面 i 个字符和序列 2 中前面 j 个字符的最大相似值，则有如下的表达式成立：

```
f[i][j]=f[i-1][j-1]+ w[s1[i]-'a'][s2[j]-'a'];       //表示不插入*的情况
f[i][j] = max(f[i][j], f[i - 1][j] + w[s1[i]-'a'][26]);     //在序列 2 中插入*
f[i][j] = max(f[i][j], f[i][j - 1] + w[26][s2[j]-'a']);     //在序列 1 中插入*
```

其中 w 表示序列 1 第 i 个字符和序列 2 第 j 个字符的差值。

第 3 个测试用例的添加方法（上下都可以添加*），结果是 5 ：

```
*e*bc
aez*c
```

递推过程如下：

f[i][j]

	0	1	2	3
	空	e	b	c
0 空	0	-5	-10	-15
1 a	-5	-4	-6	-11
2 e	-10	5	0	-5
3 z	-15	0	-5	-10
4 c	-20	-5	-1	5

f[4][3]=5

$$f[1][1]=\max\begin{cases} f[0][0]+\frac{e}{a}=-4 \\ f[0][1]+(-5)=\begin{matrix} e & * \\ * & a \end{matrix}=-10 \\ f[1][0]+(-5)=\begin{matrix} * & e \\ a & * \end{matrix}=-10 \end{cases}=-4$$

$$f[1][2]=\max\begin{cases} f[0][1]+\frac{b}{a}=-5-1=-6 \quad \begin{pmatrix} e & b \\ * & a \end{pmatrix} \\ f[0][2]+(-5)=-15 \quad \begin{pmatrix} e & b & * \\ * & * & a \end{pmatrix} \\ f[1][1]+(-5)=-9 \quad \begin{pmatrix} e & b \\ a & * \end{pmatrix} \end{cases}=-6$$

$$f[1][3]=\max\begin{cases} f[0][2]+\frac{c}{a}=-10+(-2)=-12 \quad \begin{pmatrix} e & b & c \\ * & * & a \end{pmatrix} \\ f[0][3]+(-5)=-15-5=-20 \\ f[1][2]+(-5)=-6-5=-11 \quad \begin{pmatrix} e & b & c \\ * & a & * \end{pmatrix} \end{cases}=-11$$